

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Task 1: Activation Functions
```

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def tanh(x):
    return np.tanh(x)

def relu(x):
    return np.maximum(0, x)
```

```
# Test with a single float
```

```
x = 2.0

print("Sigmoid(", x, ") =", sigmoid(x))
print("Tanh(", x, ") =", tanh(x))
print("ReLU(", x, ") =", relu(x))
```

```
# Task 2: Plot Sigmoid, Tanh and ReLU
```

```
x_values = np.linspace(-10, 10, 400)

plt.figure(figsize=(10,6))

plt.plot(x_values, sigmoid(x_values), label="Sigmoid")
plt.plot(x_values, tanh(x_values), label="Tanh")
plt.plot(x_values, relu(x_values), label="ReLU")

plt.title("Activation Functions")

plt.xlabel("Input")
plt.ylabel("Output")

plt.grid(True)
```

```
plt.legend()
plt.show()
```

Task 3: Vanishing Gradient Problem

```
def sigmoid_derivative(x):
    s = sigmoid(x)
    return s * (1 - s)

print("\nSigmoid Derivatives:\n")
inputs = np.linspace(-10, 10, 21)
for value in inputs:
    print(f"x = {value:5.1f}  derivative = {sigmoid_derivative(value):.6f}")
```

Plot derivative

```
plt.figure(figsize=(10,5))
plt.plot(x_values, sigmoid_derivative(x_values))
plt.title("Sigmoid Derivative (Vanishing Gradient)")
plt.xlabel("Input")
plt.ylabel("Derivative")
plt.grid(True)
plt.show()
```

Task 4: Dying ReLU Demonstration

```
def dying_relu_demo():
    negative_inputs = np.array([-10, -8, -6, -4, -2, -1])
    outputs = relu(negative_inputs)

    print("\nNegative Inputs:")
```

```
print(negative_inputs)
print("\nReLU Outputs:")
print(outputs)
dying_relu_demo()
```

Task 5: Modified Sigmoid with Scaling Parameter

```
def scaled_sigmoid(x, a=1):
    return 1 / (1 + np.exp(-a*x))

plt.figure(figsize=(10,6))
plt.plot(x_values, sigmoid(x_values),
         label="Original Sigmoid (a=1)")
plt.plot(x_values, scaled_sigmoid(x_values, a=0.5),
         label="Scaled Sigmoid (a=0.5)")
plt.plot(x_values, scaled_sigmoid(x_values, a=2),
         label="Scaled Sigmoid (a=2)")
plt.title("Original vs Scaled Sigmoid")
plt.xlabel("Input")
plt.ylabel("Output")
plt.grid(True)
plt.legend()
plt.show()
```

Explanation

Task 1: Activation Functions

Sigmoid

Formula

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Output range: **0 to 1**
 - Used in Binary Classification
-

Tanh

Formula

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Output range: **-1 to 1**
 - Zero-centered
-

ReLU

Formula

$$ReLU(x) = \max(0, x)$$

- Negative values become **0**
 - Positive values remain unchanged
 - Most commonly used activation function in deep learning.
-

Task 2

The graph displays:

- **Blue:** Sigmoid
- **Orange:** Tanh
- **Green:** ReLU

Observations:

- Sigmoid forms an **S-shaped curve**.
 - Tanh also forms an **S-shape**, but ranges from **-1 to 1**.
 - ReLU outputs **0 for negative inputs** and increases linearly for positive inputs.
-

Task 3: Vanishing Gradient Problem

Derivative of sigmoid:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Near $x = 0$, the derivative is approximately **0.25**, which is its maximum.

For very large positive or negative values:

- $x = -10 \rightarrow \text{derivative} \approx 0$
- $x = 10 \rightarrow \text{derivative} \approx 0$

This means gradients become extremely small, causing the **vanishing gradient problem**. During backpropagation, very small gradients make weight updates tiny, so deep neural networks learn very slowly.

Task 4: Dying ReLU Problem

Example:

Input

[-10 -8 -6 -4 -2 -1]

Output

[0 0 0 0 0 0]

Since all negative values become **0**, the neuron outputs zero and its gradient is also zero. If this continues during training, the neuron may never activate again. This is called the **Dying ReLU Problem**.

Task 5: Scaled Sigmoid

Modified function:

$$f(x) = \sigma(ax)$$

where **a** is a scaling factor.

- **a = 0.5** → smoother, flatter curve.
- **a = 1** → original sigmoid.
- **a = 2** → steeper curve, reaches 0 and 1 more quickly.